UTA | SUMO ROBOTICS CUP

Informe de trabajo del backend

Cambios implementados, verificaciones y trabajo que debe completar el frontend

Fecha de la revision	20 de octubre de 2026
Proyecto	UTA SUMO ROBOTICS CUP
Rama revisada	feature/backend-core
Alcance de hoy	Backend de combate en vivo, integridad de pruebas y documentacion tecnica
SMTP	Se mantiene opcional y pendiente de credenciales del operador

Este documento registra los cambios del backend realizados y verificados en esta jornada. Distingue lo que ya responde desde la API de lo que el frontend todavia debe integrar; no afirma que la interfaz completa este terminada.

Documento tecnico de entrega

1. Resumen ejecutivo

Se completo el contrato de combate actual indicado para la API: consulta publica, control del estado y del marcador por personal autorizado, y distribucion de cambios a clientes WebSocket. El marcador se guarda en SQL Server dentro de la transaccion y el WebSocket notifica solo despues del commit.

Durante la verificacion aparecio un fallo real en una prueba de compatibilidad del middleware de carga. Se restauro la compatibilidad con su constructor anterior y se agregaron pruebas unitarias especificas para el contrato del combate y el ciclo de suscripcion WebSocket.

El backend ya ofrece otras capacidades del proyecto - autenticacion, inscripciones, pagos manuales, reportes, reglamentos, catalogos y administracion - descritas en la guia tecnica general. El trabajo de hoy no rehizo esos modulos; verifico el conjunto de pruebas y actualizo las referencias de migraciones para incluir el esquema vigente.

Resultado de verificacion: 32 pruebas aprobadas con SQL Server local habilitado; 0 fallos y 0 omitidas en esa corrida. La API respondio estado saludable y la prueba HTTP/WebSocket del combate concluyo con limpieza de sus datos temporales.

2. Cambios realizados hoy en backend

2.1 API del combate actual

- **GET publico:** `/api/v1/competition/combate-actual`. Devuelve evento, ronda, categoria, robots, equipos, instituciones, estado, puntajes y fecha de actualizacion. Sin seleccion activa devuelve `SIN_COMBATE_ACTIVO` con puntajes en cero.
- **PUT protegido:** la misma ruta requiere JWT con rol `ADMIN` o `EVENT_MANAGER`. Puede seleccionar un partido programado, cambiar estado o fijar ambos puntajes en una sola peticion.
- Acepta los nombres del contrato en espanol (`estado`, `puntaje_a`, `puntaje_b`, `motivo`) y tambien los equivalentes en ingles definidos por el esquema.
- Valida que se seleccionen exactamente dos equipos aprobados y que el robot enviado corresponda a ese equipo, evento y categoria. Si un equipo tiene varios robots aprobados, exige seleccionar el robot.
- El marcador recibido es absoluto; SQL persiste solo la diferencia en movimientos auditables de `ScoreEntries`. Exige actualizar los dos lados juntos y un motivo no vacio.
- Valida transiciones de estado del combate/evento y evita puntuar eventos cerrados o equipos ajenos al partido. La seleccion, historial, estado y movimientos de puntos se confirman atomica y transaccionalmente.

2.2 WebSocket en vivo

- Ruta publica exacta: `ws://127.0.0.1:8000/ws/combate-actual`.
- Al conectarse, cada cliente recibe una instantanea actual; las actualizaciones enviadas por el PUT o por los endpoints existentes de estado/puntuacion se transmiten tras guardar el commit.
- El administrador de conexiones envia a varios clientes, usa concurrencia asincrona y elimina sockets desconectados o fallidos.
- El administrador WebSocket es local al proceso. Un despliegue con varios workers requiere un broker compartido; esta limitacion queda documentada y no impide operar el backend local con un worker.

2.3 Integridad, pruebas y documentacion

- Se hizo opcional el parametro `max_regulation_bytes` del middleware, manteniendo compatible la inicializacion anterior que solo indicaba el limite de comprobantes.
- Se agregaron pruebas unitarias del esquema de actualizacion (alias en espanol, estados validos, puntajes en pareja y motivo obligatorio) y del administrador WebSocket (instantanea inicial, broadcast y desconexion).
- README del backend y guia general actualizados para enumerar las migraciones 06-20, los cambios del combate y el numero actualizado de pruebas.
- No se modifico el codigo del frontend en esta jornada.

3. Base de datos y migraciones

Se incorporaron dos migraciones SQL aditivas; no requieren eliminar ni recrear la base y preservan partidos y puntuaciones existentes.

Migracion	Cambio	Uso
`19_current_combat.sql`	Crea la referencia singleton `ActiveCombat` que identifica el partido seleccionado y `ActiveCombatHistory` para auditar seleccion, reemplazo, estado y marcador.	Permite que GET y WebSocket consulten una sola fuente persistida para el combate que se muestra en vivo.
`20_match_team_robot.sql`	Agrega `RobotId` y `TeamPosition` a los equipos de un partido, con claves, restricciones e indice.	Fija robot y lado A/B del enfrentamiento aunque el equipo tenga mas de un robot aprobado.

El ejecutor `scripts/migrate.py` ya incluye ambas migraciones y las aplica con control de checksum y transaccion por migracion. En esta sesion se comprobo la base local: el evento 1 quedo en estado DRAFT, sin combate activo y sin rondas de prueba residuales.

El README ahora describe el modelo fuente mas las migraciones aditivas 10-20 y registra que las migraciones locales 06-20 estan aplicadas. La ejecucion en un ambiente productivo requiere primero respaldar y validar su destino; la base productiva no fue tocada.

Datos que no se guardan en estas tablas: conexiones del WebSocket, estado de conexion de clientes, envio SMTP y secretos. Las credenciales y tokens no aparecen en las respuestas del marcador.

4. Contrato para integrar el frontend

Consulta inicial del marcador

```
{
  "active": false,
  "id_combate": null,
  "match_id": null,
  "robot_a": null,
  "equipo_a": null,
  "puntaje_a": 0,
  "robot_b": null,
  "equipo_b": null,
  "puntaje_b": 0,
  "estado": "SIN_COMBATE_ACTIVO"
}
```

Seleccionar e iniciar un partido existente

Metodo PUT, encabezado `Authorization: Bearer <token>`; el usuario debe ser ADMIN o EVENT_MANAGER.

```
{
  "match_id": 12,
  "estado": "EN_VIVO",
  "motivo": "Inicio confirmado por el juez"
}
```

Actualizar el marcador

```
{
  "puntaje_a": 2,
  "puntaje_b": 1,
  "motivo": "Punto validado por el juez"
}
```

Estados que entiende el endpoint: `PROGRAMADO`, `EN_VIVO`, `PAUSADO`, `FINALIZADO` y `CANCELADO`. Para WebSocket el navegador debe conectarse por protocolo `ws://` en local y `wss://` cuando se despliegue detras de HTTPS.

La asignacion lado A/lado B viene de `TeamPosition`. El consumidor debe usar esos campos en cada actualizacion recibida y no inferir el lado por orden alfabetico o por identificador.

5. Verificacion ejecutada

Comprobacion	Resultado
Pruebas unitarias y de reglas	21 aprobadas; incluyen autenticacion base, validadores, reglas de competencia y pruebas nuevas del contrato del combate y WebSocket.
Integracion SQL Server	11 aprobadas con fixtures transaccionales revertidos al finalizar.
Corrida conjunta	32 passed, 0 failed, 0 skipped con RUN_SQLSERVER_TESTS=1.
Compilacion Python	compileall sobre app, routers, scripts y tests: correcta.
Formato/diff	git diff --check: sin errores de espacios ni conflictos de formato.
Health check	GET /api/v1/health: status ok; SQL Server conectado.

Prueba HTTP/WebSocket del combate realizada contra el backend local y SQL Server

Accion	Respuesta observada
GET publico sin combate	200; estado SIN_COMBATE_ACTIVO.
PUT sin JWT / con rol participante	401 sin autenticacion y 403 con rol PARTICIPANT.
Crear ronda y partido temporales	201 para ambos; equipos y robots aprobados.
Seleccionar/iniciar combate	200; estado EN_VIVO. El WebSocket recibio el cambio.
Actualizar puntaje a 2-1	200; GET y WebSocket reflejaron 2-1.
Finalizar combate	200; estado FINALIZADO transmitido por WebSocket.
Limpieza de datos de prueba	Se eliminaron ronda, partido, puntajes e historial de prueba. Se restauro el estado original del evento a DRAFT.

La suite SQL usa una transaccion externa revertida. La prueba manual del WebSocket usa registros temporales y los elimina en un bloque de limpieza; se verifico luego que no existiera combate activo ni registros con prefijo codex-smoke.

6. Trabajo que falta en el frontend para consumir el backend

Prioridad 1 - Autenticacion administrativa real

- Reemplazar el acceso simulado de `LoginAdmin.jsx` (credencial fija `eventix` y token `mock_token_...`) por POST `/api/v1/auth/login` con correo y contrasena.
- Guardar el access token de forma consistente, consultar `GET /api/v1/auth/me`, permitir unicamente roles reales y enviar al login ante 401. Mostrar 403 sin esconder el error.
- Implementar guard de rutas: `ADMIN` y `EVENT_MANAGER` pueden ver el panel operativo; solo ADMIN debe recibir funciones financieras y de administracion global.

Prioridad 2 - Retirar mocks del panel y conectar operaciones

- Reemplazar las inscripciones estaticas/localStorage de `adminInscripcionesApi.js` por `GET /api/v1/admin/registrations` con filtros y paginacion desde SQL.
- Conectar revision de inscripcion y pago con los endpoints de transicion; enviar motivos cuando el backend los exige y volver a cargar el estado confirmado por servidor.
- Mostrar comprobantes desde la ruta autorizada; manejar errores, estados de carga, vacios, rechazo de archivo y reintentos.

Prioridad 3 - Marcador y resultados oficiales

- En `resultadosApi.js` sustituir localStorage/polling simulado por GET inicial a `/api/v1/competition/combate-actual` y suscripcion al WebSocket `/ws/combate-actual`.
- Adaptar `Resultados.jsx` a los nombres reales: `robot_a`, `institucion_a`, `equipo_a`, `puntaje_a`, sus equivalentes B, `estado` y `updated_at`.
- El panel de arbitraje debe listar/seleccionar un partido existente y enviar PUT con token y motivo. El marcador debe reflejar solo la respuesta confirmada por el servidor.
- Agregar reconexion con espera gradual, indicador de perdida de conexion y refresco GET al recuperar WebSocket; no usar datos inventados como fuente oficial.

Prioridad 4 - Reportes y reglamentos

- Crear una seccion de reportes consumiendo `/api/v1/reports/overview`, `/registrations/by-status`, `/by-category`, `/trend` y los endpoints `/details` paginados.
- Mostrar importes/resumen de pagos y `/payments/details` solamente para ADMIN. Event manager recibe estadisticas operativas sin cifras financieras.
- Incorporar lectura del reglamento publico por evento/categoria y descarga del PDF publicado. El panel de administrador debe permitir listar, cargar, publicar y archivar versiones.

Prioridad 5 - Formulario y pagos

- Leer la configuracion publica del evento y tarifas actuales; presentar metodos e instrucciones configurados, consultar el reglamento aplicable y registrar la aceptacion cuando el diseno de UX lo incluya.
- El flujo actual debe mostrar la orden PDF generada, permitir cargar PDF/PNG/JPEG de comprobante y mostrar que el pago queda pendiente de verificacion administrativa; no confundir recibo de solicitud con factura fiscal o pago confirmado.
- Verificar que todos los errores de API se muestren con mensajes usables y que los identificadores de evento, categoria, equipo, robot y pago coincidan con las respuestas del backend.

7. Configuracion externa y limites conocidos

Elemento	Situacion al cierre	Accion de operador/equipo
SMTP	Excluido expresamente del trabajo de hoy. La configuracion y el outbox quedan en codigo; sin servidor y credenciales el correo no se puede enviar.	Agregar credenciales posteriormente en backend .env y reiniciar el servicio; ejecutar una prueba con buzones controlados.
Google Identity	El backend ya valida ID tokens y utiliza GOOGLE_CLIENT_ID; no se encontro Client ID local configurado.	Crear/verificar el OAuth Client ID en Google Cloud, configurar GOOGLE_CLIENT_ID en backend y VITE_GOOGLE_CLIENT_ID en frontend, autorizar el origen local y probar cuenta real.
Frontend local	frontend/uta-sumo-web/.env no existe al momento de esta revision. El codigo cae al API base local por defecto.	Crear .env con VITE_API_BASE_URL y, cuando exista, VITE_GOOGLE_CLIENT_ID; reiniciar Vite.
Almacenamiento privado	API configura directorios locales para comprobantes y reglamentos; el archivo no se sirve estaticamente.	En despliegue montar volumen privado persistente, respaldar datos y archivos juntos, restringir permisos y validar restauracion.
WebSocket en escala	ConnectionManager vive en memoria del proceso; funciona y fue probado con una instancia/worker.	Si se usan varios procesos o replicas, incorporar broker compartido (p. ej. Redis Pub/Sub) y probar broadcast entre replicas antes de escalar.
Pagos	El backend registra tarifa/monto, comprobante y revision de personal; no valida movimientos bancarios automaticamente.	Si el evento requiere tarjeta o confirmacion bancaria automatica, seleccionar e integrar pasarela/webhook con idempotencia.

El identificador de Google, los secretos de produccion y las credenciales de correo son configuracion de operadores; no se deben incorporar a commits ni a este PDF. El archivo local backend .env contiene claves y no se copia a la documentacion.

8. Lista de cierre para la siguiente fase

Backend

- Mantener y aplicar las migraciones 19 y 20 en cada ambiente antes de iniciar la version que usa combate actual.
- No ejecutar reset de la base en datos reales; respaldar SQL y almacenamiento antes de desplegar migraciones.
- Cuando se prepare despliegue multinodo, agregar broker para WebSocket; mientras tanto usar un worker por instancia.
- Probar configuracion real de Google y SMTP cuando el operador proporcione los datos; el correo quedo expresamente fuera de esta entrega.

Frontend

- Eliminar tokens y usuarios simulados del login administrativo y no conservar mocks como fallback silencioso en produccion.
- Integrar autenticacion, panel de inscripciones/pagos, reportes paginados, reglamentos y resultados/WebSocket.
- Construir y probar la aplicacion con roles ADMIN, EVENT_MANAGER y PARTICIPANT; verificar permisos, errores, expiracion y cierre de sesion.
- Configurar entorno Vite, completar flujo Google si se desea y probar registro, inscripcion, comprobante, revision y consulta de resultados desde navegador.

Comandos de verificacion backend en PowerShell

```
cd backend/uta_sumo_api
.venv\Scripts\python.exe scripts\migrate.py
$env:RUN_SQLSERVER_TESTS = '1'
.venv\Scripts\python.exe -m pytest -q
Remove-Item Env:\RUN_SQLSERVER_TESTS
.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Postman: importar `backend/uta_sumo_api/postman/UTA_Sumo_API.postman_collection.json` y el environment local correspondiente. La coleccion describe GET/PUT del combate; la conexion WebSocket se crea como solicitud WebSocket independiente en `ws://127.0.0.1:8000/ws/combate-actual`. No almacenar tokens ni contrasenas en archivos versionados.

Estado final del alcance de hoy: backend actualizado, migraciones 19-20 aplicadas localmente y suite completa aprobada. La interfaz frontend no se modifico y aun necesita integrar las rutas listadas en la seccion 6.